COMP2048: Theory of Computing

Dr. Kasra Khosoussi

Week 2

5/3/2026

Assignment 1 on Blackboard

Almost half of class has not visited Ed yet.

About Ed Discussion Board

- Important announcements and useful student questions are posted on Ed.
- Please check **Ed, Blackboard, and your email** regularly.
- You can post questions **privately on Ed** – only the teaching staff will see them.
- **Assignment Questions:** Please make your question **private** if your Ed post might reveal your solution to an assignment problem (or if you're unsure).
- If appropriate (and you have no objection), we can make a question **public (anonymously)** so the whole class can benefit.

Contact Us :)

- If you have any specific/personal concerns that you'd like to privately discuss with Josh or myself, please feel free to reach out to schedule a meeting.

  comp2048@eecs.uq.edu.au

- Please ask all other general questions (about lectures, assignments, admin, etc) on Ed (publicly/privately/anonymously) or during applied classes.

Today's Plan

- Continue with DFA
- Regular Languages
- Closure Properties of Regular Languages
- Regular Expressions
- Non-deterministic Finite Automata (NFA)

Recap

- Alphabet Σ
- Set of all strings Σ^*
- Decision Problem

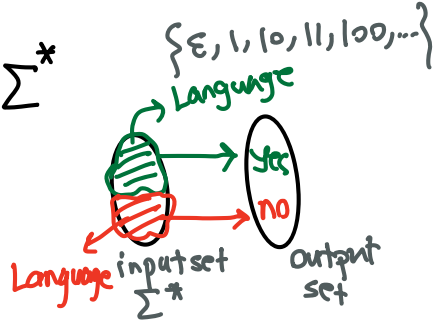
Example $\Sigma = \{0, 1\}$

$\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, \ldots\}$

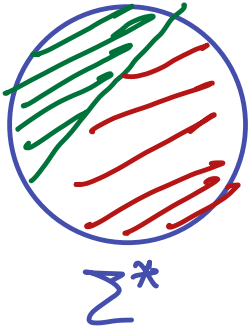
f : Input set $\rightarrow$ Output set $= \{yes, no\}$

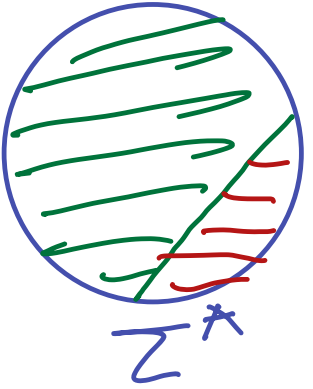
encoding inputs as strings: $\Sigma^* \rightarrow \{yes, no\}$

- Languages : subsets of Σ^*
- Languages $\Leftrightarrow$ Decision Problems (1-to-1 correspondence)



Recap





Recap: Deterministic Finite Automata (DFA)

A DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states
- Σ is the input alphabet
- $\delta : Q \times \Sigma \to Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of accept states

Designing DFAs: General Strategy

- DFA has no memory other than its current state.
- When designing a DFA, ask yourself:

  "What is the minimal amount of information I need to remember about the string read so far to accept/reject the string at the end?"
- Each state corresponds to a specific piece of remembered information.
- You must define a transition for **every** state and **every** symbol in Σ.

Example 1: Strings Ending in '1'

Build a DFA for the language of strings over $\Sigma = \{0,1\}$ that **end with a 1**

What do we need to remember?

We only care about the *last* symbol we saw.

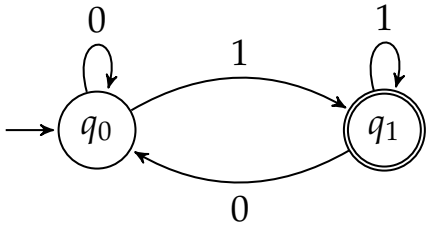
- State q_0: The last symbol was 0 (or we haven't read anything yet).
- State q_1: The last symbol was 1 (this is our accept state!)

Example 1: Strings Ending in '1'

Build a DFA for the language of strings over $\Sigma = \{0,1\}$ that **end with a 1**

What do we need to remember?
We only care about the *last* symbol we saw.

- State q_0: The last symbol was 0 (or we haven't read anything yet).
- State q_1: The last symbol was 1 (this is our accept state!)



Example 2: The "Trap" State

Build a DFA for strings over $\Sigma = \{a,b\}$ that **start with the letter 'a'**.

What do we need to remember?

We only care about the *very first* symbol.

- State q_0: Start state (haven't read anything).
- State q_a: The first symbol was a (Accept! We stay here forever).
- State q_{trap}: The first symbol was b (Reject! The string is ruined, and no future input can fix it).

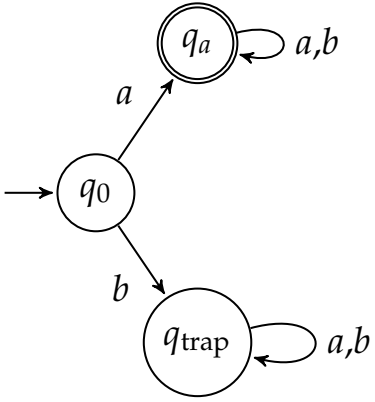
Example 2: The "Trap" State

Build a DFA for strings over $\Sigma = \{a,b\}$ that **start with the letter 'a'**.

What do we need to remember?

We only care about the *very first* symbol.

- State q_0: Start state (haven't read anything).
- State q_a: The first symbol was a (Accept! We stay here forever).
- State q_{trap}: The first symbol was b (Reject! The string is ruined, and no future input can fix it).



Example 3: Containing a Substring

Build a DFA for strings over $\Sigma = \{0,1\}$ that **contain the substring "00"**.

What do we need to remember?

How far along are we in seeing the pattern "00"?

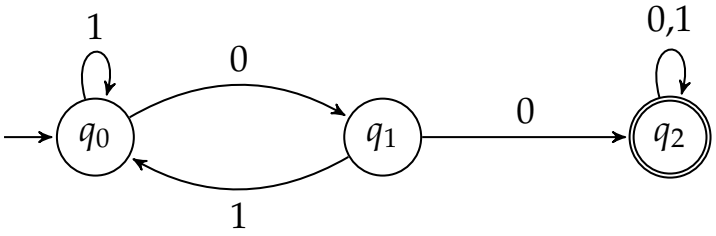
- State q_0: Haven't seen a 0 recently.
- State q_1: Saw one 0 (if we see a 1 now, our progress resets to q_0).
- State q_2: Saw "00"! (Accept! We stay here forever, because the string now contains "00").

Example 3: Containing a Substring

Build a DFA for strings over $\Sigma = \{0,1\}$ that **contain the substring "00"**.

What do we need to remember?
How far along are we in seeing the pattern "00"?

- State q_0: Haven't seen a 0 recently.
- State q_1: Saw one 0 (if we see a 1 now, our progress resets to q_0).
- State q_2: Saw "00"! (Accept! We stay here forever, because the string now contains "00").



The Language of a DFA

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA.

A string $w \in \Sigma^*$ is accepted if the machine, processes w and finishes in a state in F (i.e., one of the Accepting states).

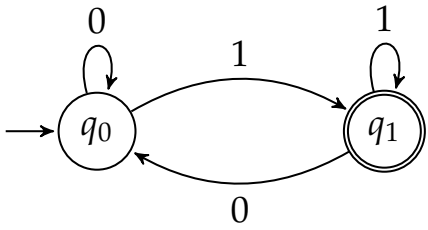
Language recognized by M

$$L(M) = \{ w \in \Sigma^* \mid M \text{ accepts } w \}$$

We say:

- $L(M)$ is recognized by DFA M
- $L(M)$ is the language of DFA M

Example



- $L = \{w \in \{0,1\}^* \mid w \text{ ends with } 1\}$ is *recognized* by this DFA
- Language of this DFA is L

Regular Languages

Definition

Language $L \subseteq \Sigma^*$ is **regular** if there exists a DFA M such that $L = L(M)$

(i.e., L is **regular** if & only if **there exists a DFA** that **recognizes** L)

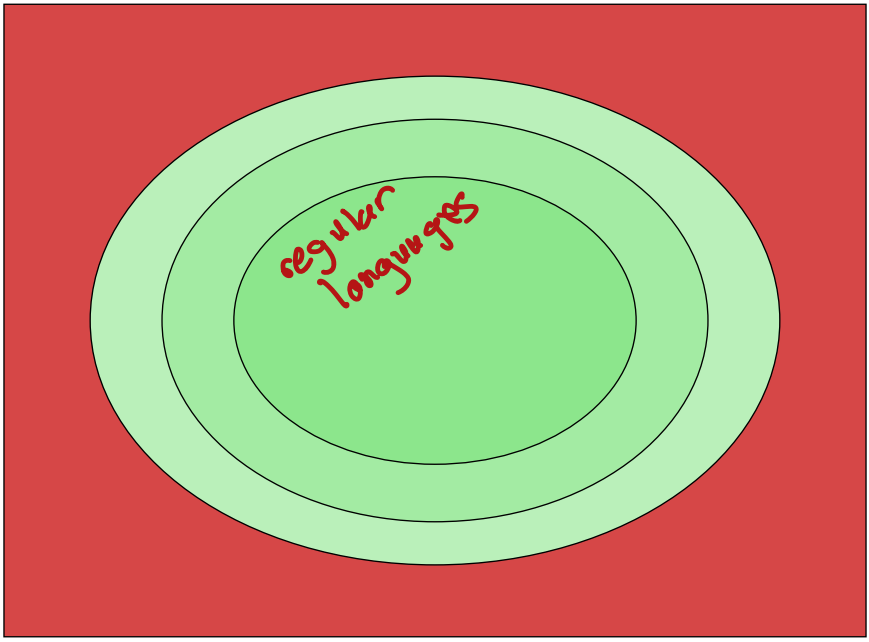
not regular → $\{ 0^n 1^n \mid n \geq 0 \}$

Examples:

- binary strings with an even number of 1's
- binary strings that contain substring 00
- binary strings ending in 1

$\underbrace{0 \ldots 0}_{n}$ $\underbrace{1 \ldots 1}_{n}$

Our Big Picture Revisited



Rectangle: set of all decision problems over Σ

Closure Properties

Theorem: If L and L' are regular languages then the following languages are also regular:

- Complement: $\overline{L} \triangleq \Sigma^* - L$
- Union: $L \cup L'$
- Intersection: $L \cap L'$
- Concatenation: $L \circ L'$ (or LL') defined as

$$LL' \triangleq \left\{ w \in \Sigma^* \mid w = w_1 w_2 \text{ such that } w_1 \in L, w_2 \in L' \right\}$$

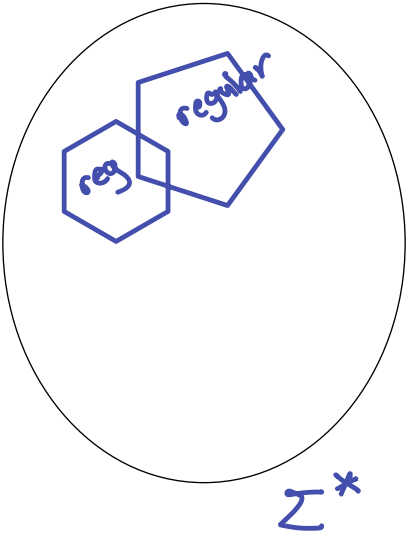
- Star: L^* defined as[1]

$$L^* \triangleq \{\varepsilon\} \cup \{w_1 \dots w_n \mid n \geqslant 1, w_i \in L \text{ for } i = 1, \dots, n\}$$

[1] Similar to Kleene star that we apllied to **alphabets** . . . the set of all strings obtained by concatentating a finite number of (**zero** or more) strings from L.

Why is this useful?

- We can build complex regular languages from simpler ones
- We can build complex DFA from simpler ones



Proof Sketch: Closure Under Complement

Prove **if L is regular, then $\overline{L}$ is regular**

- Since L is regular, there exists a DFA M that recognizes L.
- We must show there exists a DFA M' that recognizes $\overline{L}$.
- Let $M = (Q, \Sigma, \delta, q_0, F)$
- How to construct M' from M?
- $M' = (Q, \Sigma, \delta, q_0, Q - F)$, i.e., flipping Accepting and Reject states

can you explain why M' recognizes $\overline{L}$?

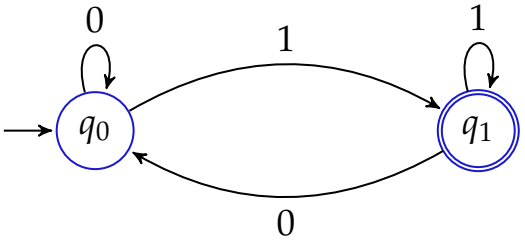
Proof Sketch: Closure Under Complement

Prove **if L is regular, then $\overline{L}$ is regular**

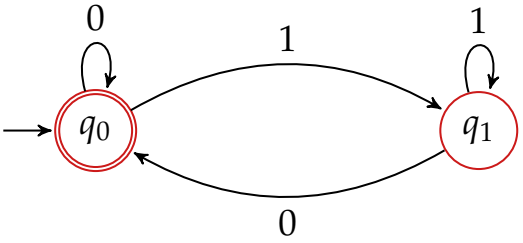
- Since L is regular, there exists a DFA M that recognizes L.
- We must show there exists a DFA M' that recognizes $\overline{L}$.
- Let $M = (Q, \Sigma, \delta, q_0, F)$
- How to construct M' from M?
- $M' = (Q, \Sigma, \delta, q_0, Q - F)$, i.e., flipping Accepting and Reject states

can you explain why M' recognizes $\overline{L}$?

Example: DFA for L



DFA for $\overline{L}$



Proof Sketch: Closure Under Union & Intersection

Prove **if** L **and** L' **are regular, then** $L \cup L'$ **and** $L \cap L'$ **are regular**

- L and L' are regular

  $\Leftrightarrow \exists^2$ DFAs M and M' that recognize L and L', respectively:

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$M' = (Q', \Sigma, \delta', q_0', F')$$

Our plan

use these DFAs to construct DFAs recognizing union & intersection

[2] There exists ...

Proof Sketch: Closure Under Union & Intersection

- **Idea:** On any given input string:
  - Run M and M'
  - Accept the input when:
    - **Union:** M **or** M' (or both) finish in an Accepting state
    - **Intersection:** M **and** M' finish in an Accepting state

How to do this with DFAs?

Proof Sketch: Closure Under Union & Intersection

- **Idea:** On any given input string:

  - Run M and M'

  - Accept the input when:
    - **Union:** M **or** M' (or both) finish in an Accepting state
    - **Intersection:** M **and** M' finish in an Accepting state

How to do this with DFAs?

Proof Sketch: Closure Under **Union**

- $M = (Q, \Sigma, \delta, q_0, F)$ recognizes L
- $M' = (Q', \Sigma, \delta', q_0', F')$ recognizes L'

Construct DFA M_{union} for $L \cup L'$:

- States: $Q_\times = Q \times Q'$
- Start state: (q_0, q_0')
- Transition function:
  $\forall$ symbol $a \in \Sigma$ and $(q, q') \in Q_\times$

$$\delta_\times\left((q, q'), a\right) = (\delta(q, a), \delta'(q', a))$$

- Accepting states:

$$F_{\text{union}} = \{(q, q') \mid q \in F \text{ or } q' \in F'\}$$

M_{union} accepts strings either M or M' accept (or both)

$$M_{\text{union}} \triangleq \left(Q_\times, \Sigma, \delta_\times, (q_0, q_0'), F_{\text{union}}\right)$$

Proof Sketch: Closure Under **Intersection**

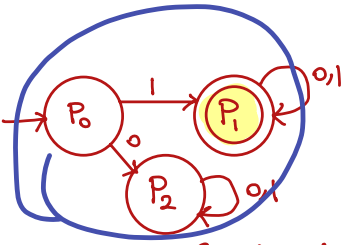
- Same idea except for the Accepting states:

$$F_{\text{inter}} = \{(q, q') \mid q \in F \textbf{ and } q' \in F'\}$$

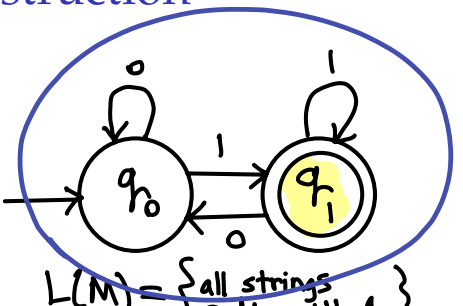
M_{inter} accepts string accepted by both M and M'

$$M_{\text{inter}} \triangleq \left(Q_{\times}, \Sigma, \delta_{\times}, (q_0, q_0'), F_{\text{inter}}\right)$$

Example of the Product Construction



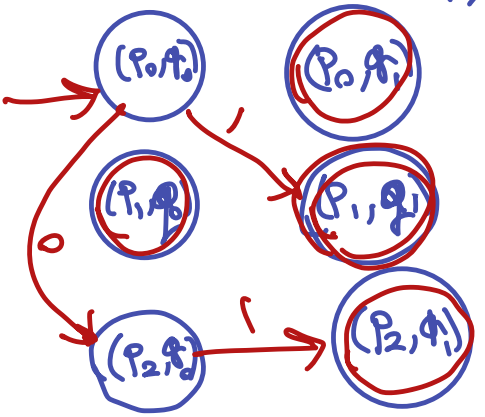
$L(N) = \{$ all strings starting with 1 $\}$



$L(M) = \{$ all strings ending with 1 $\}$

$Q \times$

$L(N) \cup L(M)$



Regular Expressions

Regular Expressions

Regular expressions (regex) are another way to describe regular languages (we may prove this).

Base expressions:

- $\emptyset$
- ε
- Single symbol from the alphabet (e.g., a for $a \in \Sigma$)

Regular Operations: If R and S are regex:

- $R \cup S$ (union, "or")
- RS (concatenation)
- R^* ("zero or more of")

Order of operations:

(i) parentheses, (ii) star, (iii) concatenation, (iv) union

Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

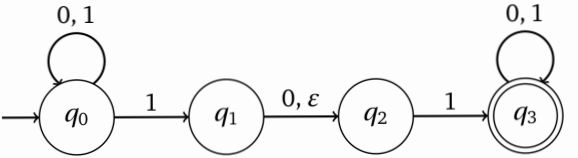
Regex Examples

Over $\Sigma = \{0,1\}$:

- $(0 \cup 1)^*$: all binary strings
- $1(0 \cup 1)^*$: strings starting with 1
- $(0 \cup 1)^*00(0 \cup 1)^*$: strings containing 00
- $0^*(10^*10^*)^*0^*$: **think about this one!**

Non-deterministic Finite Automata

Non-determinism



In DFA:

- From each state and for each symbol there is **exactly one** transition
- Each transition consumes a symbol

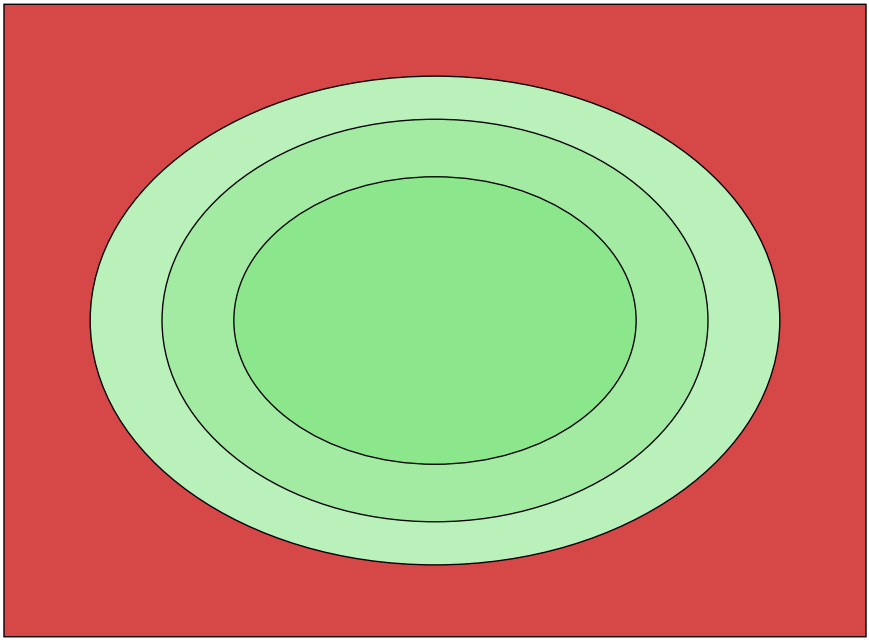
What if we allow:

- zero, one, or more possible next states in each transition
- transitions without consuming input ε-transitions

$\Rightarrow$ can be in **multiple states** after reading kth symbol from input

does non-determinism make our model more powerful?

Our Big Picture Revisited

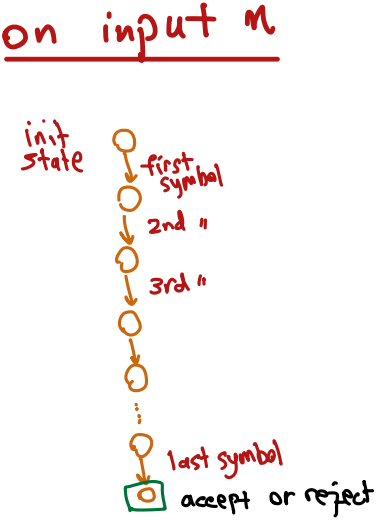


Rectangle: set of all decision problems over Σ

Review: Deterministic Finite Automata (DFA)

A DFA is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

- Q is a finite set of states
- Σ is the input alphabet
- $\delta : Q \times \Sigma \to Q$ is the transition function
- $q_0 \in Q$ is the start state
- $F \subseteq Q$ is the set of accept states



Non-deterministic Finite Automata (NFA)

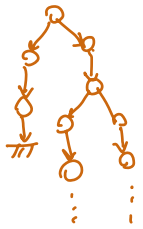
A **Non-deterministic Finite Automaton (NFA)** is like a DFA but:

- $\delta : Q \times \Sigma \cup \{\epsilon\} \to 2^Q$
  (may go to **zero or more states** upon reading a symbol)[3]
- Can have ε-transitions (transitions without consuming input)
- **Acceptance Rule:** Accepts a string if *any branch* (that consumes the entire input string from start to end) leads to an accept state

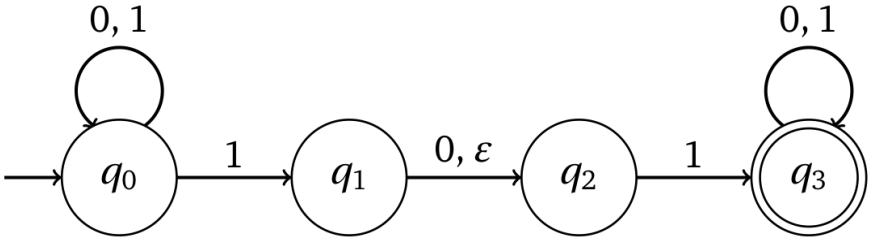
Example:

- $\delta(q_2, a) = \emptyset$
- $\delta(q_3, b) = \{q_1, q_5\}$
- $\delta(q_4, \varepsilon) = \{q_1, q_4, q_5\}$

on input a



[3] We can have $0, 1, \ldots$ outgoing edges from state q upon reading symbol a. 2^Q or $\mathcal{P}(Q)$ is called the power set and denotes the set of all subsets of Q; i.e., $2^Q = \{R \mid R \subseteq Q\} = \{\emptyset, \{q_0\}, \{q_1\}, \ldots, \{q_n\}, \{q_0, q_1\}, \ldots, \{q_0, q_1, \ldots, q_n\}\}$.



$\delta(q_0, 1) = \{q_0, q_1, q_2\}$

$\delta(q_2, 0) = \emptyset$

$\delta(q_1, \varepsilon) = \{q_2\}$

not consume symbol

Acceptance in an NFA

An NFA accepts a string w if

there exists a computation path[4]

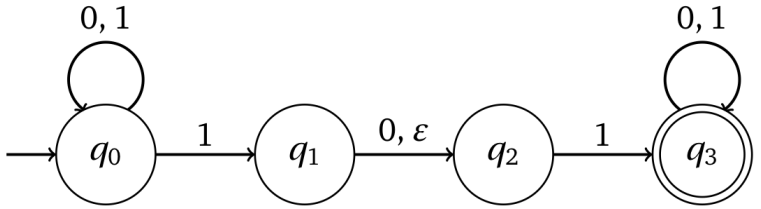
that leads from the start state to an accepting state

Equivalently:

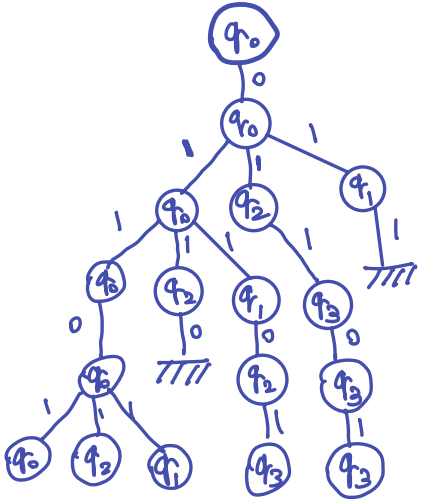
- if at least one possible state after reading w is in $F \Rightarrow$ **Accept**
- otherwise $\Rightarrow$ **Reject**

[4] i.e., sequence of valid transitions, starting from the initial state

Example NFA



$x = 01101$



NFA for Union

NFA for Concatenation

NFA for Star